

NAME: RAMYA S S

REG NO: 192421226

SLOT - D

COURSE: CSA1407 COMPILER DESIGN

CASE STUDY

SET 1

Q1. Parse Tree / Syntax Tree Representation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 int main() { 3 printf("Expression: a + b * c\n\n"); 4 5 printf("Parse Tree:\n"); 6 printf(" +\n"); 7 printf(" / \ 8 printf(" a *\n"); 9 printf(" / \ 10 printf(" b c\n"); 11 12 printf("\nSyntax Tree (AST):\n"); 13 printf(" +\n"); 14 printf(" / \ 15 printf(" a *\n"); 16 printf(" / \ 17 printf(" b c\n"); 18 19 printf("\nComparison:\n"); 20 printf("Parse Tree : More nodes, higher memory usage.\n"); 21 printf("Syntax Tree: Fewer nodes, easier traversal and optimization.\n"); 22 printf("AST is mainly useful in semantic analysis and code generation.\n"); 23 24 return 0; 25 }</pre>		Expression: a + b * c Parse Tree: <pre> + / \ a * / \ b c</pre> Syntax Tree (AST): <pre> + / \ a * / \ b c</pre> Comparison: Parse Tree : More nodes, higher memory usage. Syntax Tree: Fewer nodes, easier traversal and optimization. AST is mainly useful in semantic analysis and code generation. === Code Execution Successful ===

Q2. Bottom-Up Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 5, b = 3, result; 5 6 result = a + b; 7 printf("Top-Down: %d + %d = %d\n", a, b, result); 8 9 result = a + b; 10 printf("Bottom-Up: %d + %d = %d\n", a, b, result); 11 12 printf("\nTop-Down: Easy semantic actions, grammar restrictions.\n"); 13 printf("Bottom-Up: Efficient parsing, handles more grammars.\n"); 14 15 return 0; 16 }</pre>		Top-Down: 5 + 3 = 8 Bottom-Up: 5 + 3 = 8 Top-Down: Easy semantic actions, grammar restrictions. Bottom-Up: Efficient parsing, handles more grammars. === Code Execution Successful ===

Q3. Attribute Dependency

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5, c = 2; 5 int result; 6 7 result = a + b * c; 8 9 printf("Expression: a + b * c\n"); 10 printf("Result: %d\n", result); 11 12 printf("\nEvaluation order: b*c -> a+(b*c)\n"); 13 printf("Dependency graph ensures correct order.\n"); 14 printf("Precedence and associativity avoid conflicts.\n"); 15 16 return 0; 17 }</pre>		Expression: a + b * c Result: 20 Evaluation order: b*c -> a+(b*c) Dependency graph ensures correct order. Precedence and associativity avoid conflicts. === Code Execution Successful ===

Q4. Static Type Checking

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 int result = a + b; 6 7 printf("a = %d, b = %d\n", a, b); 8 printf("Result = %d\n", result); 9 printf("Static typing detects type errors during compilation.\n"); 10 printf("It improves safety and software reliability.\n"); 11 12 return 0; 13 }</pre>		a = 10, b = 5 Result = 15 Static typing detects type errors during compilation. It improves safety and software reliability. === Code Execution Successful ===

Q5. Function Overloading / Type Resolution Demonstration

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int addInt(int a, int b) { 4 return a + b; 5 } 6 7 float addFloat(float a, float b) { 8 return a + b; 9 } 10 11 int main() { 12 int a = 10, b = 5; 13 float x = 2.5, y = 1.5; 14 15 printf("Integer: %d\n", addInt(a, b)); 16 printf("Float: %.1f\n", addFloat(x, y)); 17 18 printf("Compiler resolves types based on function arguments.\n"); 19 printf("Advanced types increase complexity and ambiguity.\n"); 20 21 return 0; 22 }</pre>		Integer: 15 Float: 4.0 Compiler resolves types based on function arguments. Advanced types increase complexity and ambiguity. === Code Execution Successful ===

SET 2

Q1. Synthesized Attribute

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5, c = 2; 5 int result; 6 7 // Bottom-up evaluation 8 int temp = b * c; 9 result = a + temp; 10 11 printf("Expression: a + b * c\n"); 12 printf("Synthesized value: %d\n", result); 13 printf("Evaluation: b*c -> a+(b*c)\n"); 14 printf("Bottom-up evaluation is suitable.\n"); 15 16 return 0; 17 }</pre>		<pre>Expression: a + b * c Synthesized value: 20 Evaluation: b*c -> a+(b*c) Bottom-up evaluation is suitable. === Code Execution Successful ===</pre>

Q2. Type Conversion

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10; 5 float b = 2.5; 6 float result; 7 8 result = a + b; // int converted to float 9 10 printf("Integer: %d\n", a); 11 printf("Float: %.1f\n", b); 12 printf("Result: %.1f\n", result); 13 14 printf("Type conversion ensures compatibility.\n"); 15 printf("Type equivalence helps detect valid assignments.\n"); 16 17 return 0; 18 }</pre>		<pre>Integer: 10 Float: 2.5 Result: 12.5 Type conversion ensures compatibility. Type equivalence helps detect valid assignments. === Code Execution Successful ===</pre>

Q3. Syntax Tree and Bottom-Up Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5, c = 2; 5 6 // Compact syntax tree: a + (b * c) 7 int temp = b * c; 8 int result = a + temp; 9 10 printf("Expression: a + b * c\n"); 11 printf("Syntax Tree: +(a, *(b,c))\n"); 12 printf("Bottom-up result: %d\n", result); 13 14 printf("Syntax trees use less memory than parse trees.\n"); 15 printf("S-attributed definitions work well with LR parsing.\n"); 16 17 return 0; 18 }</pre>		<pre>Expression: a + b * c Syntax Tree: +(a, *(b,c)) Bottom-up result: 20 Syntax trees use less memory than parse trees. S-attributed definitions work well with LR parsing. === Code Execution Successful ===</pre>

Q4. Left-to-Right Attribute Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 int temp, result; 6 7 // Left-to-right evaluation 8 temp = a + b; 9 result = temp * 2; 10 11 printf("Expression: (a + b) * 2\n"); 12 printf("Intermediate code:\n"); 13 printf("t1 = a + b\n"); 14 printf("t2 = t1 * 2\n"); 15 printf("Result = %d\n", result); 16 17 printf("L-attributed evaluation: left to right.\n"); 18 printf("Dependency management ensures correct order.\n"); 19 20 return 0; 21 }</pre>		<pre>Expression: (a + b) * 2 Intermediate code: t1 = a + b t2 = t1 * 2 Result = 30 L-attributed evaluation: left to right. Dependency management ensures correct order. === Code Execution Successful ===</pre>

Q5. Type Checking

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10; 5 float b = 5.5; 6 7 float x = a; // int to float 8 int y = (int)b; // float to int 9 10 printf("Integer: %d\n", a); 11 printf("Float: %.1f\n", b); 12 printf("Int to Float: %.1f\n", x); 13 printf("Float to Int: %d\n", y); 14 15 printf("Type checking ensures compatibility.\n"); 16 printf("Type equivalence improves reliable compilation.\n"); 17 18 return 0; 19 }</pre>		<pre>Integer: 10 Float: 5.5 Int to Float: 10.0 Float to Int: 5 Type checking ensures compatibility. Type equivalence improves reliable compilation. === Code Execution Successful ===</pre>

SET 3

Q1. Syntax-Directed Definition – Expression Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // S-attributed: bottom-up evaluation 8 int sum = a + b; 9 10 // L-attributed: left-to-right evaluation 11 float result = sum * c; 12 13 printf("Syntax Tree: *(+(a,b),c)\n"); 14 printf("S-attributed result: %d\n", sum); 15 printf("L-attributed result: %.2f\n", result); 16 printf("Type conversion: int -> float\n"); 17 18 return 0; 19 }</pre>		<pre>Syntax Tree: *(+(a,b),c) S-attributed result: 15 L-attributed result: 37.50 Type conversion: int -> float === Code Execution Successful ===</pre>

Q2. Type Checker with Type Conversion

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 float result = a + (b * c); 8 9 printf("Syntax Tree: +(a,*(b,c))\n"); 10 printf("Result: %.2f\n", result); 11 printf("Type Checker: int converted to float.\n"); 12 printf("Valid types improve compiler reliability.\n"); 13 14 return 0; 15 }</pre>		<pre>Syntax Tree: +(a,*(b,c)) Result: 22.50 Type Checker: int converted to float. Valid types improve compiler reliability. === Code Execution Successful ===</pre>

Q3. Bottom-Up Expression Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // Bottom-up evaluation 8 int sum = a + b; 9 float result = sum * c; // int converted to float 10 11 printf("Type System: int + int = int\n"); 12 printf("Type Conversion: int -> float\n"); 13 printf("Bottom-Up Result: %.2f\n", result); 14 printf("Bottom-up evaluation generates code efficiently.\n"); 15 16 return 0; 17 }</pre>		<pre>Type System: int + int = int Type Conversion: int -> float Bottom-Up Result: 37.50 Bottom-up evaluation generates code efficiently. === Code Execution Successful ===</pre>

Q4. Type Equivalence Checking

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // Syntax-directed evaluation 8 int sum = a + b; 9 float result = sum * c; 10 11 printf("Syntax Tree: *(+(a,b),c)\n"); 12 printf("Evaluated Result: %.2f\n", result); 13 14 // Type equivalence and conversion 15 if (sizeof(a) == sizeof(b)) 16 printf("Type Equivalence: Compatible\n"); 17 18 printf("Type Conversion: int -> float\n"); 19 printf("Semantic errors are reduced.\n"); 20 21 return 0; 22 }</pre>		<pre>Syntax Tree: *(+(a,b),c) Evaluated Result: 37.50 Type Equivalence: Compatible Type Conversion: int -> float Semantic errors are reduced. === Code Execution Successful ===</pre>

Q5. Type Checking and Type Conversion

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // Bottom-up evaluation 8 int sum = a + b; 9 float result = sum * c; 10 11 printf("Top-Down: Simple but complex for nested dependencies.\n"); 12 printf("Bottom-Up: Efficient for complex expressions.\n"); 13 printf("Syntax Tree: *+(a,b),c)\n"); 14 15 printf("Type Checking: int and float are compatible.\n"); 16 printf("Type Conversion: int -> float\n"); 17 printf("Result: %.2f\n", result); 18 19 return 0; 20 }</pre>		Top-Down: Simple but complex for nested dependencies. Bottom-Up: Efficient for complex expressions. Syntax Tree: *+(a,b),c) Type Checking: int and float are compatible. Type Conversion: int -> float Result: 37.50 === Code Execution Successful ===

SET 4

Q1. Syntax-Directed Definition and Type Conversion

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // Syntax-directed evaluation 8 int sum = a + b; 9 float result = sum * c; 10 11 printf("Syntax Tree: *+(a,b),c)\n"); 12 printf("Result: %.2f\n", result); 13 14 // Type conversion 15 float x = a; 16 printf("Type Conversion: int -> float = %.1f\n", x); 17 18 printf("Syntax-directed definitions build syntax trees.\n"); 19 printf("Type conversion ensures compatible operations.\n"); 20 21 return 0; 22 }</pre>		Syntax Tree: *+(a,b),c) Result: 37.50 Type Conversion: int -> float = 10.0 Syntax-directed definitions build syntax trees. Type conversion ensures compatible operations. === Code Execution Successful ===

Q2. Top-Down and Bottom-Up Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float x = 2.5; 6 7 // Bottom-up evaluation 8 int sum = a + b; 9 float result = sum * x; 10 11 printf("Top-Down: Starts from root and moves downward.\n"); 12 printf("Bottom-Up: Evaluates operands and moves upward.\n"); 13 printf("Result: %.2f\n", result); 14 15 // Type equivalence 16 if (sizeof(a) == sizeof(b)) 17 printf("Type Equivalence: Compatible\n"); 18 19 printf("Type checking prevents unsafe assignments.\n"); 20 21 return 0; 22 }</pre>		Top-Down: Starts from root and moves downward. Bottom-Up: Evaluates operands and moves upward. Result: 37.50 Type Equivalence: Compatible Type checking prevents unsafe assignments. === Code Execution Successful ===

Q3. Inherited and Synthesized Attributes

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // Synthesized attribute: bottom-up result 8 int sum = a + b; 9 10 // Type checking and conversion 11 float result = sum * c; 12 13 printf("Synthesized: %d\n", sum); 14 printf("Inherited: context passed to expressions\n"); 15 printf("Type Conversion: int -> float\n"); 16 printf("Result: %.2f\n", result); 17 printf("Type checking prevents invalid operations.\n"); 18 19 return 0; 20 }</pre>		<pre>Synthesized: 15 Inherited: context passed to expressions Type Conversion: int -> float Result: 37.50 Type checking prevents invalid operations. === Code Execution Successful ===</pre>

Q4. Type Equivalence and Type Conversion

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // Syntax tree: a + (b * c) 8 float result = a + (b * c); 9 10 printf("Syntax Tree: +(a,(b,c))\n"); 11 printf("Result: %.2f\n", result); 12 13 printf("Type Equivalence: int and float are compatible.\n"); 14 printf("Type Conversion: int -> float\n"); 15 printf("Conversion improves compiler accuracy.\n"); 16 17 return 0; 18 }</pre>		<pre>Syntax Tree: +(a,(b,c)) Result: 22.50 Type Equivalence: int and float are compatible. Type Conversion: int -> float Conversion improves compiler accuracy. === Code Execution Successful ===</pre>

Q5. S-Attributed and L-Attributed Definitions

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 float c = 2.5; 6 7 // S-attributed: bottom-up evaluation 8 int sum = a + b; 9 10 // L-attributed: left-to-right evaluation 11 float result = sum * c; 12 13 printf("S-Attributed: %d\n", sum); 14 printf("L-Attributed: %.2f\n", result); 15 printf("Type Conversion: int -> float\n"); 16 printf("Type System: checks type compatibility.\n"); 17 18 return 0; 19 }</pre>		<pre>S-Attributed: 15 L-Attributed: 37.50 Type Conversion: int -> float Type System: checks type compatibility. === Code Execution Successful ===</pre>

SET 5

Q1. L-Attributed Definition

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5, result; 5 6 // Top-down / left-to-right evaluation 7 int temp = a + b; 8 result = temp * 2; 9 10 printf("Expression: (a+b)*2\n"); 11 printf("Syntax Tree: *+(a,b),2)\n"); 12 printf("L-Attributed Result: %d\n", result); 13 printf("Evaluation: left-to-right\n"); 14 printf("Inherited attributes maintain context.\n"); 15 16 return 0; 17 }</pre>		<pre>Expression: (a+b)*2 Syntax Tree: *+(a,b),2) L-Attributed Result: 30 Evaluation: left-to-right Inherited attributes maintain context. === Code Execution Successful ===</pre>

Q2. Bottom-Up Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5, c = 2; 5 6 // Bottom-up evaluation 7 int temp = b * c; 8 int result = a + temp; 9 10 printf("Syntax Tree: +(a,*(b,c))\n"); 11 printf("Bottom-Up Result: %d\n", result); 12 printf("S-Attributed: synthesized attributes\n"); 13 printf("LR Parsing: child values -> parent\n"); 14 15 return 0; 16 }</pre>		<pre>Syntax Tree: +(a,*(b,c)) Bottom-Up Result: 20 S-Attributed: synthesized attributes LR Parsing: child values -> parent === Code Execution Successful ===</pre>

Q3. Type Equivalence

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a[2][2] = {{1,2},{3,4}}; 5 int b[2][2]; 6 7 b[0][0] = a[0][0]; 8 9 printf("Type A: int[2][2]\n"); 10 printf("Type B: int[2][2]\n"); 11 printf("Structural Equivalence: Compatible\n"); 12 printf("Assignment: Valid\n"); 13 14 return 0; 15 }</pre>		<pre>Type A: int[2][2] Type B: int[2][2] Structural Equivalence: Compatible Assignment: Valid === Code Execution Successful ===</pre>

Q4. Type Checking and Type Conversion

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10; 5 float b = 2.5; 6 float result; 7 8 // Type conversion: int -> float 9 result = a + b; 10 11 printf("Integer: %d\n", a); 12 printf("Float: %.1f\n", b); 13 printf("Result: %.1f\n", result); 14 printf("Type Checking: Compatible\n"); 15 printf("Type Conversion: int -> float\n"); 16 17 return 0; 18 }</pre>		<pre>Integer: 10 Float: 2.5 Result: 12.5 Type Checking: Compatible Type Conversion: int -> float === Code Execution Successful ===</pre>

Q5. Attribute Dependency and Expression Evaluation

main.c	Run	Output
<pre>1 #include <stdio.h> 2 3 int main() { 4 int a = 10, b = 5; 5 int temp, result; 6 7 // Bottom-up evaluation 8 temp = a + b; 9 10 // Top-down / Left-to-right dependency 11 result = temp * 2; 12 13 printf("Syntax Tree: *(+(a,b),2)\n"); 14 printf("Bottom-Up Result: %d\n", result); 15 printf("L-Attributed: Left-to-right evaluation\n"); 16 printf("Dependency: a+b -> temp -> result\n"); 17 18 return 0; 19 }</pre>		<pre>Syntax Tree: *(+(a,b),2) Bottom-Up Result: 30 L-Attributed: Left-to-right evaluation Dependency: a+b -> temp -> result === Code Execution Successful ===</pre>